Now the output is *'Hi undefined You joined in undefined'*. Basically, the fields *this.name* and *this.joinyear* are now undefined. Why does this happen? In the above code snippet, when we invoked *bad_call*, the *'this'* does not point to *Employee*. Instead, it points to the 'global object' and not the *Employee* object. The fields *name* and *joinyear* are undefined in the 'global object', and hence you get the output for their values as *undefined*. Therefore, in JavaScript, the context pointed to by *'this'* changes, depending on how a function is called. Let us next take a quick look at closures.

Closures in JavaScript

```
function greet(){
       var name = "sandya";
       function greet_by_name() {
                    alert ("hello " + name);
       }
        greet_by_name();
}
```

In the above code snippet, there is an inner function *greet_by_name*, which is defined inside an outer function *greet*. The variable *name* inside the inner function will match the variable *name* defined in the closest enclosing scope—which, in this case, is the variable *name* defined in the outer function, *greet*. Therefore, we can print the name as *'sandya'* in the inner function *greet_by_name*. Now let us consider a slight modification to the above code:

```
function greet(){
       var name = "sandya";
       return  function greet_by_name() {
                    alert ("hello " + name);
       };
}
//This is where the closure is created
var greet_sandya = greet();
//the closure function is invoked
greet_sandya();
```

If you run the above code, you will see the same output as before. However, note that we returned a function *greet_by_name* from the function *greet*. So when we executed the statement *'var greet_sandya = greet();'*, we only executed *greet*, and assigned the return value *greet_by_name* to the variable *greet_sandya*. We did not actually execute the function *greet_by_name*.

When we executed *greet_sandya* in the next line, we executed *greet_by_name*, since it had been assigned to the *greet_sandya* variable earlier. Now, the question is: How is *greet_sandya* able to access the variable *name* defined inside its outer function *greet* successfully, even though *greet* has already finished executing? Well, the answer lies in closures. A closure not only includes a function definition, but it also includes the environment where the function was defined. This allows *greet_sandya* to access the variables defined inside *greet* when it is actually executed. However, it is not all that simple. Since *name* is a local variable allocated on the stack of the *greet* function, how can it still be available when the function *greet* has already finished execution and is no longer active on the call stack? Well, that will be answered when we discuss closures in detail next month.

My 'must-read book' for this month

This month's must-read book recommendation comes from me, and it is for a book titled 'Cracking the Systems Software Interview', published by TMH, and co-authored by Ganesh (author of the 'Joy of Programming' column) and me. This book was in response to the requests by many of our readers for a quick refresher to prepare for systems software interviews in product companies. It covers coding questions, algorithms, operating systems, computer architecture and concurrency. It is based on our experience of conducting interviews in product MNCs over the last 10 years. Please do let me know your comments and suggestions to improve the book.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, happy programming, and here's wishing you the very best! END

By: Sandya Mannarswamy

The author is an expert in systems software, and is currently working as a researcher in IBM India Research Labs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group, 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?gid=2339182*.